Assignment 1 learning from data

Louis Cooke

2024-11-14

Question 1:

a).

We will simulate 1,000,000 values from each of the following standard Uniform variables X1 and X2.

```
set.seed(123)

n <- 1000000
X1 <- runif(n, min = 0, max = 1)
X2 <- runif(n, min = 0, max = 1)



Y <- (floor(10 * X1 + 1) + floor(10 * X2 + 1)) %% 2
```

b).

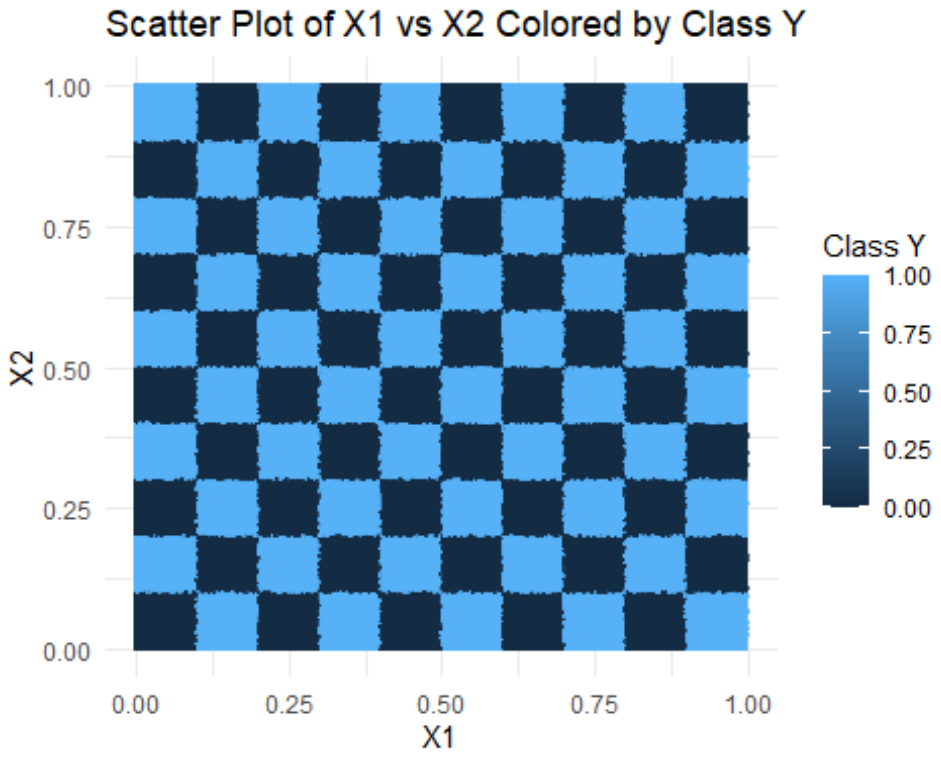
We will create a data frame of X1, X2 and Y, with Y as a factor type (for categorical data).

```
data_frame <- data.frame(y = Y, x1 = X1, x2 = X2)
data_frame$y <- as.factor(data_frame$y)
```

c).

We will create a scatter plot for data X1 and X2, using the categorical data Y to colour data points

```
ggplot(data = data_frame, mapping = aes(x = X1, y = X2, color = Y)) +
  geom_point(size = 0.5, alpha = 0.7) +
  labs(x = "X1", y = "X2", color = "Class Y") +
  ggtitle("Scatter Plot of X1 vs X2 Colored by Class Y") +
  theme_minimal()
```



d).

We are asked to simulate data for two uniform variables, X1 and X2. We use set.seed() to produce the same uniform random variables each time, for reproducibilty. The input X1 and X2 are uniform random variables between 0 and 1, when X1 and X2 are inputed into Y(x), X1 is transformed into a decimal value greater than 1 given by Z1 = (10 * X1 + 1) and likewise for X2: Z2 = (10 * X2 + 1). both Z1 and Z2 are 'floored', the floor function takes a floating point number (decimal) and truncates the value, so for Z1 and Z2 the output will be an integer. Y is then found by summing the two integers given by the floor function, and finding the remainder of the sum of the two integers when divided by 2 (modulus(2) of the sum of the two floored decimals). Therefore the output, Y will be either 1 or 0 because any integer is either divisible or not divisible by 2.

The scatter plot we generate colour classes Y outputs (0 and 1) into dark blue and light blue. As discussed above Z = (10 * X + 1) meaning that an X value generated from 0 to 1 eg. X1 = 0.123 and X2 = 0.456 will give Z1 = 2.23 and Z2 = 5.56, finding Y from these 2 values we have (floor(2.23) + floor(5.56)) %% 2 = (2 + 7) %% 2 = 1. We generalise this analysis by recognising that between X = 0 and X = 1 we can get a continuous range of decimal values, and there is a recurring pattern with what we observe on the scatter plot:

Keeping in mind:

Remark 1). X1(odd) + X2(odd) = Even

Remark 2). X1(even) + X2(even) = Even

Remark 3). X1(odd) + X2(even) = odd

Remark 4). X1(even) + X2(odd) = odd

Any two X values 0 <= X < 0.1 will eventually give us the sum of two odd numbers, (once the Z value is floored) which is an an even number hence its modulus is zero so we observe the dark blue square in the bottom left on the scatter plot. The same outcome occurs for two X values, 0.1 <= X < 0.2 we observe a dark blue square up and right from the bottom left but for the reason given by remark 2. The light blue squared, where we see a remainder of 1, can be explained by remarks 3 and 4, where generated X values produce Z values that, when floored give the sum of an odd and even number.

The distribution of the scatter plot is also uniformly distributed (equal density of data points across the plot) which is a consequence of the original distribution of X1 and X2 - uniformly distributed values between 0 and 1.

e).

Next we sample 10,000 data points out of the 1,000,000 points initially given using a random sample of a 1/100th of the data set. After we make a training and testing data split.

```
set.seed(123)
indices <- sample(1:nrow(data_frame), 10000, replace = FALSE)
sampled_data <- data_frame[indices, ] # Randomly select 10,000 points

set.seed(123)
train_indices <- sample(1:nrow(sampled_data), nrow(sampled_data) * 0.66,
replace = FALSE)
traindat <- sampled_data[train_indices, ] # Split into testing and training
data
testdat <- sampled_data[-train_indices, ]
```

f).

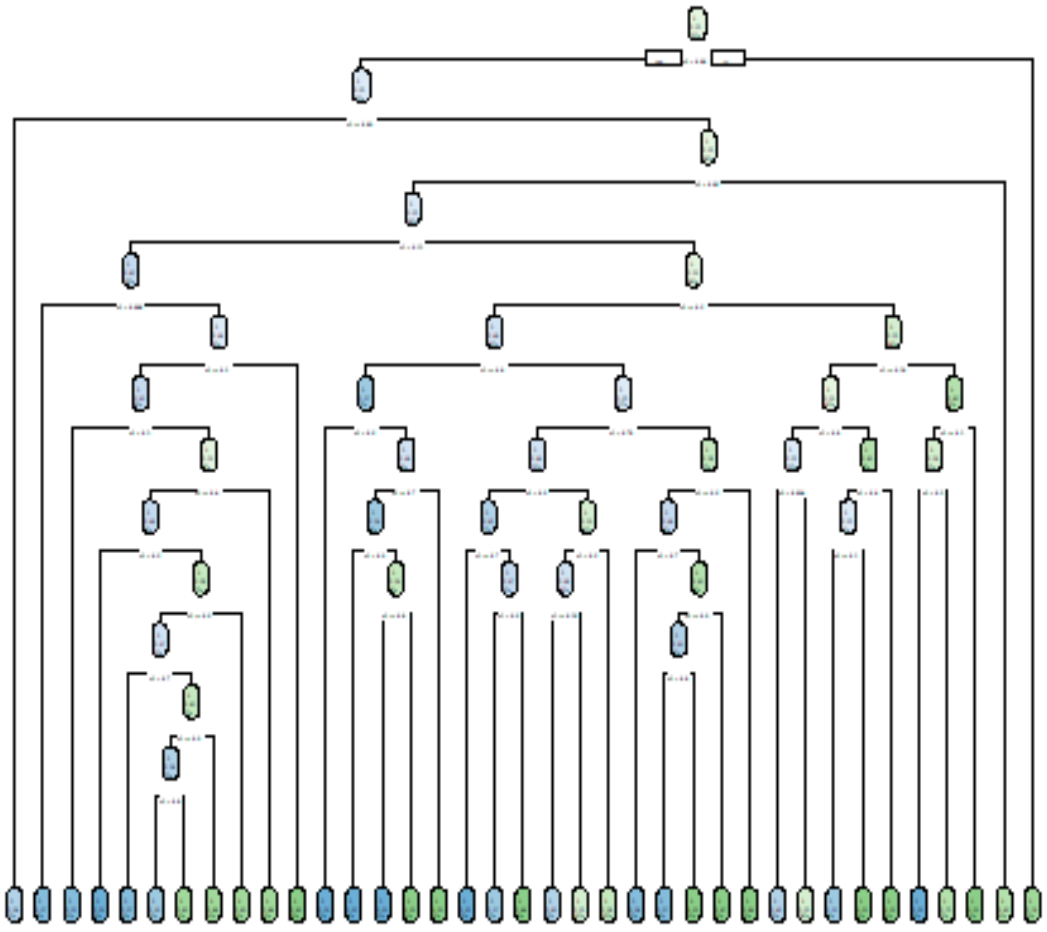
We now need to grow a classification tree. The optimal cp value was found by using the cost complexity table, from observation we can see that the cost complexity stops decreasing by a significant amount when the cp drops below 0.00809 so we chose cp=0.008 to maintain better accuracy.The classification tree for the test data was then plotted alongside its confusionMatrix() which gives insight into how well the tree performs under the test data.

We observe an accuracy of 0.6421 using the test data which means the model is succesfully predicting how to class the test data. To ensure the test data was not over-fitted we set the cost complexity such that it wasn't allowing the tree to branch further than needed. To show this the training data was also used to create predictions on the model, if the accuracy was significantly higher in the training set we can say that the model was over-fitted. The model accuracy for the training set was 0.6679, therefore we can say that the two sets make similar predictions and the model is not over-fitted.

With the cp level at an optimal level the classification tree is not over-fitted and the tree is allowed to branch out to make accurate estimates from the data. The classification tree reaches

its maximum splitting at the 14th level which, by avoiding overfitting, we can say that the tree correctly partitions the feature space. [1]

```
class_tree = rpart(y ~., data=traindat, method = "class", cp=0.008)

prediction_test_tree <- predict(class_tree, testdat, type = "class")
prediction_train_tree <- predict(class_tree, traindat, type = "class")

matrix_tree_test <- confusionMatrix(prediction_test_tree, testdat$y)
matrix_tree_train <- confusionMatrix(prediction_train_tree, traindat$y)

accuracy_tree_train <- matrix_tree_train$overall['Accuracy']
accuracy_tree_test <- matrix_tree_test$overall['Accuracy']

rpart.plot(class_tree)
```



```
print(paste("Accuracy from the training set for a classification tree was",
accuracy_tree_train))

print(paste("Accuracy from the test set for a classification tree was",
accuracy_tree_test))
```

Accuracy from the training set for a classification tree was **0.667878787878788.**

Accuracy from the test set for a classification tree was **0.642058823529412.**

g).

The random forest makes much more accurate predictions on the test set than the classification tree with an accuracy of 0.962, however the difference between accuracy of the test and training set is higher indicating that the model is slightly more overfitted than the classification tree.

For the random forest the nodesize was chosen as 10, much less than 10 would introduce overfitting allowing deeper trees and any larger, the model might miss finer details and observations produced by the data. ntree was selected as 100 which gives a balance between the computational cost of larger number of trees and the instability of a smaller number of trees. The random forest partitioned the feature space better than the classification tree by giving more accurate predictions while maintaining a good level of fitting.

```
random_forest<-rfsrc(y~., data=traindat, nodesize = 10, ntree = 100)

train_predictions_frst <- predict(random_forest, traindat)$class
test_predictions_frst <- predict(random_forest, testdat)$class

matrix_frst_train <- confusionMatrix(train_predictions_frst, traindat$y)
matrix_frst_test <- confusionMatrix(test_predictions_frst, testdat$y)

accuracy_frst_train <- matrix_frst_train$overall['Accuracy']
accuracy_frst_test <- matrix_frst_test$overall['Accuracy']

print(paste("Accuracy from the training set for a random forest was",
accuracy_frst_train))

print(paste("Accuracy from the test set for a random forest was",
accuracy_frst_test))
```

Accuracy from the training set for a random forest was **0.999545454545455.**

Accuracy from the test set for a random forest was **0.962058823529412.**

h).

The boosted model was built to maximise the accuracy of the training set while also avoiding overfitting. The parameters defined by params contained a binary logistic objective which is used for binary classification and outputs probabilities between 0 and 1, which were then converted back to test binary predictions.

The eval_metric used was a logloss error which is more suitable for the probabilistic outputs from the binary logistic objective. The maximum depth of trees was chosen to be 3, max depth below 3 produced poor accuracy of the final test data, and above 3 introduced overfitting, shown by the training set outperforming the testing set. Finally the subsample was set to 1 which minimised overfitting, allowing the model to generalise more.

The test predictions were produced using nrounds of 3400, with nrounds = 1000 we still observe moderate accuracy around 0.71 but higher nrounds allows us to see how the model performs when more weak learners are added. In the next part will show a plot that highlights the reduction in error after more and more weak learners are added to the model.

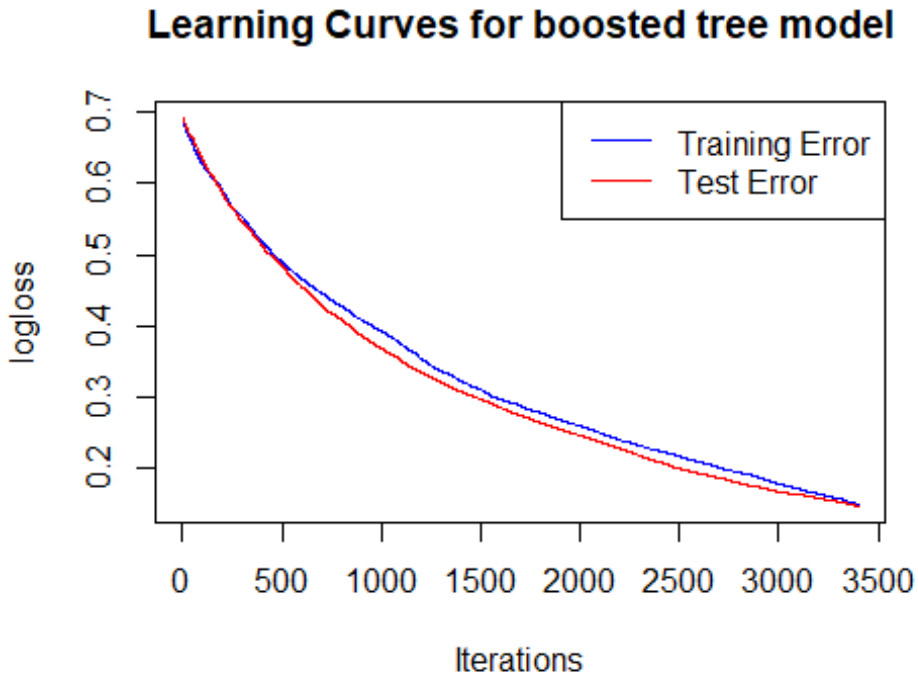
The plot shows that adding more weak learners to the model decreases the reduction in error exponentially (error reduction reduces less significantly at high nrounds), we do however observe that the training and test error lines converge near 3400 iterations suggesting that the model is leaning from the data given to it and that it is not overfitting as there is no significant gap in logloss error between the two lines.

Compared to the classification tree the boosted tree performs much better than the classification tree, when predicting the classes for the test data it has a much smaller misclassification error. The boosted tree is also less sensitive than the classification tree. With the classification tree the cp value was the predominant factor controlling how overfitted the tree was, with the boosted model the learning rate subsample and max depth were important factors that affected how well the model generalised data. While the boosted tree performed better than the classification tree, it still under performed against the random forest which had a higher accuracy. The accuracy could be controlled by increasing nrounds but came with the risk of overfitting, as we still aim to keep the number of weak learners much below the number of training data points. [2]

```
numeric_y_train = as.numeric(traindat$y) - 1
numeric_y_test = as.numeric(testdat$y) - 1

X_train <- traindat %>% select(-y)
X_test <- testdat %>% select(-y)

xgb_train <- xgb.DMatrix(data = as.matrix(X_train), label = numeric_y_train)
xgb_test <- xgb.DMatrix(data = as.matrix(X_test), label = numeric_y_test)

colnames(xgb_test)=colnames(xgb_train)
params <- list(objective = "binary:logistic", booster = "gbtree", eval_metric
= "logloss", max_depth = 3, eta = 0.1, subsample = 1, colsample_bytree = 1)

modxgb<-xgboost(data=xgb_train, params=params, nrounds=3400, verbose=0)
test_preds<-predict(modxgb,xgb_test)


# Converts probability to binary 1, 0 vals
test_preds_binary_n <- ifelse(test_preds > 0.5, 1, 0)
test_preds_binary <- as.factor(test_preds_binary_n)
matrix_boost_test = confusionMatrix(test_preds_binary, testdat$y)
accuracy_boost_test <- matrix_boost_test$overall['Accuracy']
```

```
print(paste("Accuracy from the testing set for a boosted tree was",
accuracy_boost_test))

modxgb_test<-xgboost(data=xgb_test, params=params, nrounds=3400, verbose=0)
train_errors <- modxgb$evaluation_log$train_logloss
test_errors <- modxgb_test$evaluation_log$train_logloss

plot(1:length(train_errors), train_errors, type = 'l', col = 'blue', ylim =
range(c(train_errors, test_errors)),
     xlab = 'Iterations', ylab = 'logloss', main = 'Learning Curves for
boosted tree model')
lines(1:length(test_errors), test_errors, col = 'red')
legend('topright', legend = c('Training Error', 'Test Error'), col =
c('blue', 'red'), lty = 1)
```

Accuracy from the testing set for a boosted tree was **0.86.**



i).

For the plot in the final part of question 1 we examine the effect of the parameters on the testing accuracy of the tree. For this part two parameters were examined the maximum allowed node size (maximum depth of tree) and the number of trees.

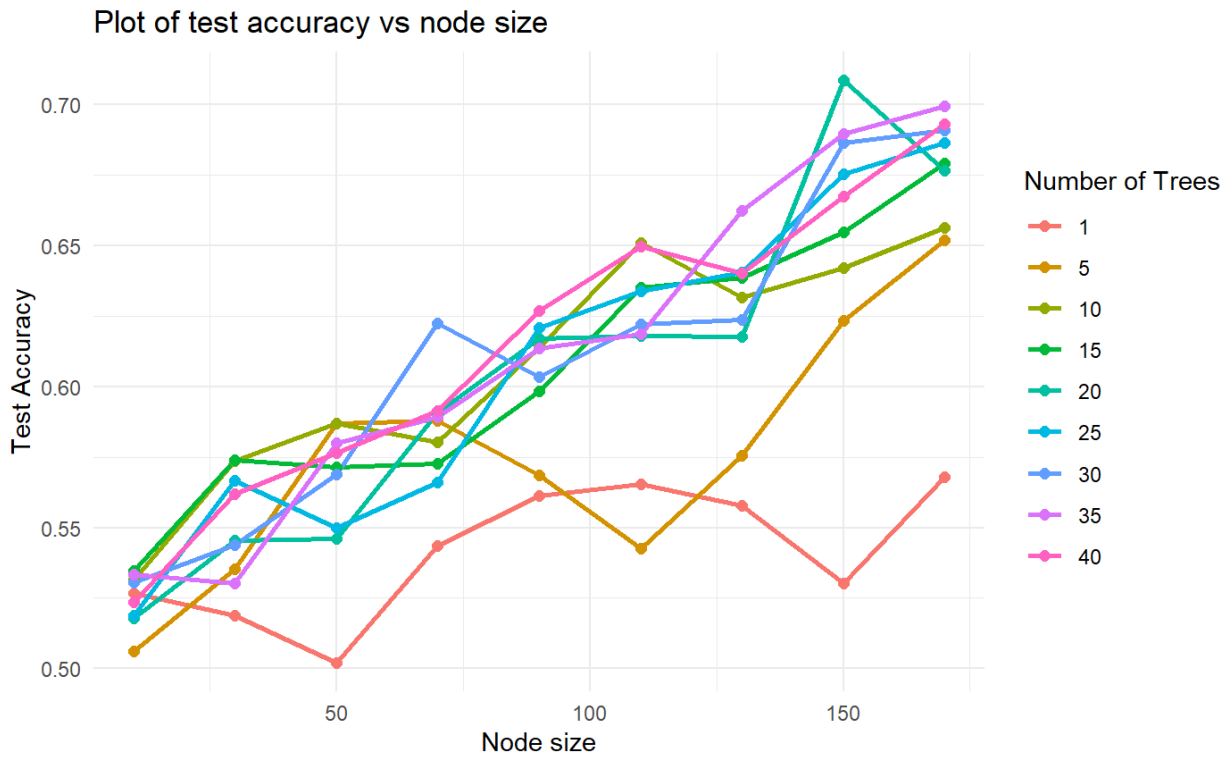
The test accuracy was positively affected by the increase in the number of trees. At lower number of trees there was a weaker relationship between node size and test accuracy. This is

likely because the model didn't have enough classification trees to learn patterns in the data set, hence there was a lack of diversity, and the model behaved more like the more inaccurate classification tree. However, for a larger number of trees the model performed better and produced more accurate results when node size increased.

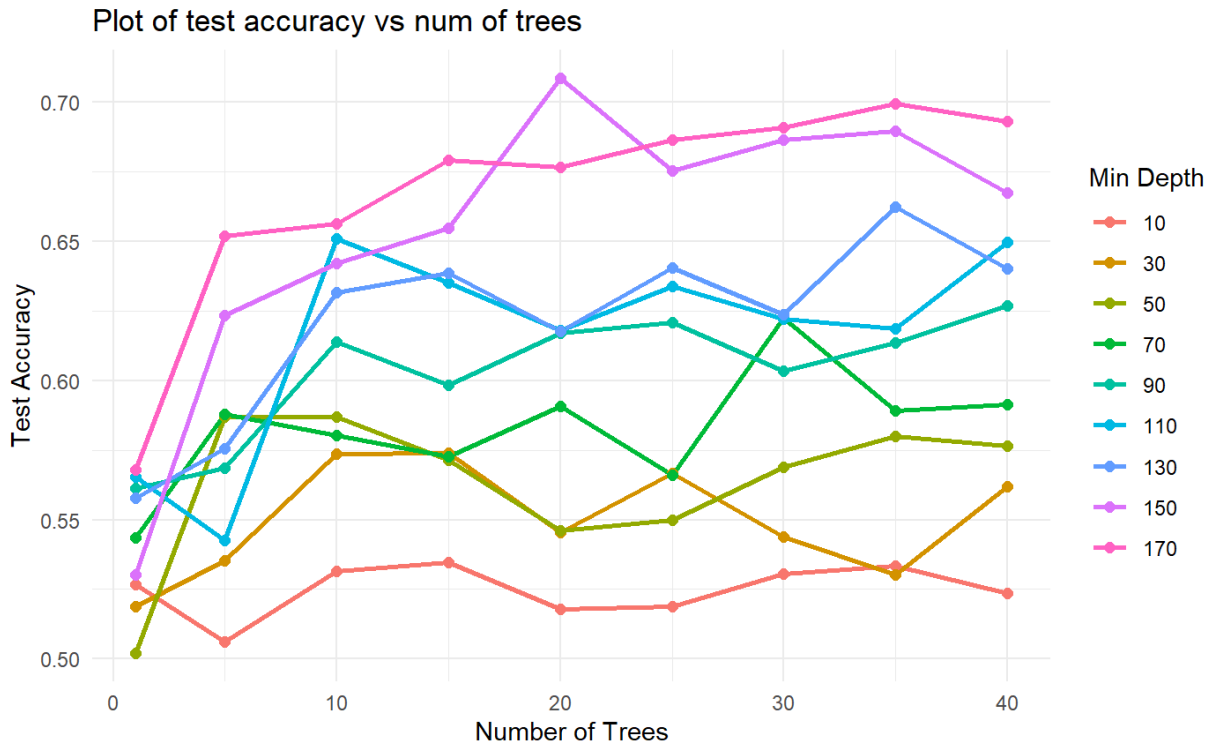
There was also a positive correlation between the number of trees and the test accuracy, where at lower minimum depth there was a weaker relationship and lower overall accuracy with increasing number of trees. This is likely because at a lower minimum depth the tree cannot diversify and learn patterns from the previous tree. Once the minimum depth increased there was a stronger positive correlation between the two. [3]

```
results <- data.frame(ntree = integer(), maxnodes = integer(), train_acc =
numeric(), test_acc = numeric())

nodesizes = c(1, 5, 10, 15, 20, 25, 30, 35, 40)
ntrees = c(10, 30, 50, 70, 90, 110, 130, 150, 170)

for (i in nodesizes) {
  for (j in ntrees) {
    rf_model <- randomForest(y ~ ., data = traindat, ntree = i, maxnodes = j)

    train_preds <- predict(rf_model, traindat)
    test_preds <- predict(rf_model, testdat)

    train_acc <- mean(train_preds == traindat$y)
    test_acc <- mean(test_preds == testdat$y)
    results <- rbind(results, data.frame(ntree = i, maxnodes = j, train_acc =
train_acc, test_acc = test_acc))
  }
}
```

```
ggplot(results, aes(x = ntree, y = test_acc, color = as.factor(maxnodes))) +
geom_line(linewidth = 1) + geom_point(size = 2) + labs(title = "Plot of test
accuracy vs num of trees", x = "Number of Trees", y = "Test Accuracy", color =
"Max Depth") + theme_minimal()
```




```
ggplot(results, aes(x = maxnodes, y = test_acc, color = as.factor(ntree))) +
geom_line(linewidth = 1) +geom_point(size = 2) +labs(title = "Plot of test
accuracy vs node size", x = "Node size", y = "Test Accuracy", color = "Number
of Trees") +theme_minimal()
```



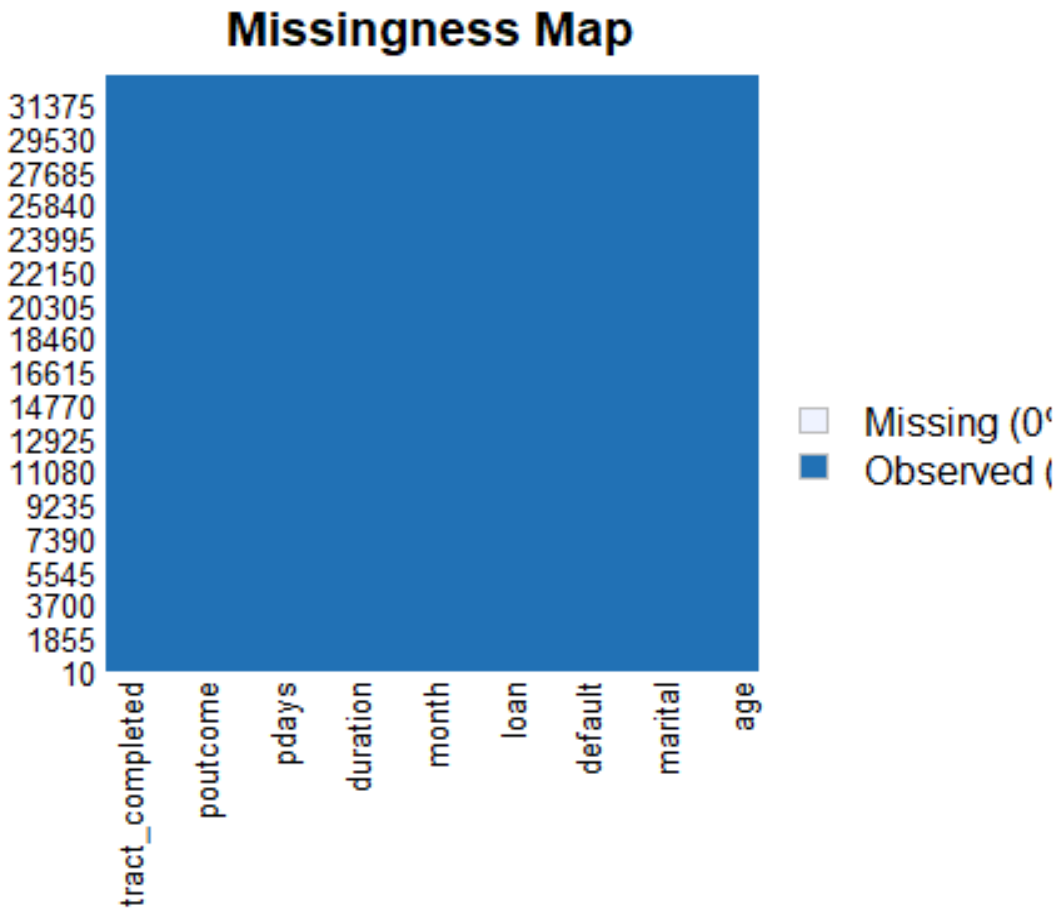
Question 2:

a).

The dataset in question is from Kaggle's Banking Dataset, which contains information about a marketing campaign of a Portuguese banking institution. The goal is to predict whether a customer will subscribe to a long-term deposit (y), based on 15 features. Performing some analysis, we find there are 10 columns of categorical data and 5 numerical data. While most of the numerical data is straightforward some requires more attention, like the number of days that passed by after the client was last contacted from a previous campaign (pdays), which has the value 999 if the client has never been contacted from a previous campaign. In this case we have a mixture of numerical and classification data.

To deal with this variable I created an extra separate binary variable - contract_completed which gives instances where pdays are 999 the value zero, and where they arent, the value 1. The target variable is a binary yes or no variable. The missingness map shows that there was no missing data from the data set.

```
bank_dat <- read.csv("Portuguese_Bank.csv", header = TRUE)

bank_dat<-bank_dat[complete.cases(bank_dat),]
bank_dat <- data.frame(bank_dat)

bank_dat$y <- factor(bank_dat$y, levels = c("no", "yes"))
bank_dat$y <- as.numeric(bank_dat$y) - 1

bank_dat$job<-as.factor(bank_dat$job)
bank_dat$marital<-as.factor(bank_dat$marital)
bank_dat$education<-as.factor(bank_dat$education)
bank_dat$default<-as.factor(bank_dat$default)
bank_dat$housing<-as.factor(bank_dat$housing)
bank_dat$loan<-as.factor(bank_dat$loan)
bank_dat$contact<-as.factor(bank_dat$contact)
bank_dat$month<-as.factor(bank_dat$month)
bank_dat$day_of_week<-as.factor(bank_dat$day_of_week)
bank_dat$poutcome<-as.factor(bank_dat$poutcome)

bank_dat$contract_completed <- ifelse(bank_dat$pdays == 999, 0, 1)
missmap(bank_dat)
```



b).

We create the training and testing data in a similar way to question 1 but since our train-test split is 2/3 we use the floor function to ensure we maintain whole values. Additionally, to ensure fair sampling we take a random sample from all the rows of the data set to generate the indices. Finally we convert the y values into factors for the models which we use after.

```
set.seed(123)
N <- nrow(bank_dat)
n <- floor(2/3*N)

indices <- sample(1:N)
train_indices <- indices[1:n]
test_indices <- indices[(n+1):N]

traindat_bank <- bank_dat[train_indices,]
testdat_bank <- bank_dat[test_indices,]

traindat_bank$y <- factor(traindat_bank$y, levels = c(0, 1))
testdat_bank$y <- factor(testdat_bank$y, levels = c(0, 1))
```

c).

For this part we will fit a logistic regression to the data to predict the binary factor variable y. From the output shown below it is clear the logistic regression is accurate in predicting the test class with a very small difference in accuracy of ~ 0.7% between the training and testing set, with the test set producing an accuracy of about 90% implying a good level of fitting.

For the actual prediction of signing a long-term deposit the model predicts about 11% at this accuracy level, found by calculating the mean of the probabilistic predictions from the test set. [4]

```
logistic_bank = glm(y ~., family = binomial(logit), data=traindat_bank)

predicted_probabilities_train <- predict(logistic_bank,
newdata=traindat_bank, type = "response")
predicted_probabilities_test <- predict(logistic_bank, newdata=testdat_bank,
type = "response")
mean_prob_logistic_bank <- mean(predicted_probabilities_test)
print(paste("The probability that a customer will sign up to a long term
deposit using a logistic regression model is", mean_prob_logistic_bank))

## [1] "The probability that a customer will sign up to a long term deposit
using a logistic regression model is 0.112694426174476"

predicted_class_train <- ifelse(predicted_probabilities_train > 0.5, 1, 0)
predicted_class_test <- ifelse(predicted_probabilities_test > 0.5, 1, 0)

predicted_class_train <- factor(predicted_class_train, levels = c(0, 1))
predicted_class_test <- factor(predicted_class_test, levels = c(0, 1))

cm_train <- confusionMatrix(predicted_class_train, traindat_bank$y)
cm_test <- confusionMatrix(predicted_class_test, testdat_bank$y)

accuracy_log_train_bank <- cm_train$overall['Accuracy']
accuracy_log_test_bank <- cm_test$overall['Accuracy']

print(paste("From the train data the probability that a customer will sign up
to a long term deposit using a logistic regression has an accuracy of",
accuracy_log_train_bank))

print(paste("From the test data the probability that a customer will sign up
to a long term deposit using a logistic regression has an accuracy of",
accuracy_log_test_bank))
```

The probability that a customer will sign up to a long-term deposit using a logistic regression model is **0.112694426174476.**

From the train data the probability that a customer will sign up to a long-term deposit using a logistic regression has an accuracy of **0.908768096148593.**

From the test data the probability that a customer will sign up to a long-term deposit using a logistic regression has an accuracy of **0.901402039329934.**

d).

The random forest model predicts a different probability of signing a long-term deposit. Shown by the accuracies of the training and test sets, we can infer that the fit generated by the random forest is not as good as the logistic regression with a difference of accuracies ~ 5% between training and test sets. The probability of signing the long term contract was also different ~ 8%.

The random forest fit was determined by the primary arguments nodesize (minimum size of leaf node) and ntree (number of trees). Increasing the nodesize seemed to have a greater affect on the accuracy difference between sets, where the accuracy of the training set mainly increased with the nodesize, showing small increase in the test set with increase in ntree. Beyond approximately ntree = 7 the model seemed to give consistent results where the increase in ntree didn't yield much higher accuracies.

So ntree was set to 10 which was sufficient for the accuracy yield while not compromising much computational cost, and nodesize was also set to 10 which was sufficiently high enough that the model could generalize patterns but not too high so that the model would miss finer details.

```
random_forest_bank <- randomForest(y~., data=traindat_bank, nodesize = 10,
ntree = 10)

train_predictions_frst_bank <- predict(random_forest_bank, traindat_bank)
test_predictions_frst_bank <- predict(random_forest_bank, testdat_bank)

matrix_frst_train_bank <- confusionMatrix(train_predictions_frst_bank,
traindat_bank$y)
matrix_frst_test_bank <- confusionMatrix(test_predictions_frst_bank,
testdat_bank$y)

accuracy_frst_train_bank <- matrix_frst_train_bank$overall['Accuracy']
accuracy_frst_test_bank <- matrix_frst_test_bank$overall['Accuracy']

test_predictions_frst_bankn <- as.numeric(test_predictions_frst_bank) - 1
mean_prob_rfst_bank <- mean(test_predictions_frst_bankn)
print(paste("The probability that a customer will sign up to a long term
deposit using a random forest model is", mean_prob_rfst_bank))
```

```
print(paste("From the train data the probability that a customer will sign up to a long term deposit using a random forest had an accuracy of:", accuracy_frst_train_bank))

print(paste("From the test data the probability that a customer will sign up to a long term deposit using a random forest had an accuracy of:", accuracy_frst_test_bank))
```

The probability that a customer will sign up to a long-term deposit using a random forest model is **0.0782957028404953.**

From the train data the probability that a customer will sign up to a long-term deposit using a random forest had an accuracy of: **0.959346262405536.**

From the test data the probability that a customer will sign up to a long-term deposit using a random forest had an accuracy of: **0.904679533867444.**

e).

Fitting a generalized additive model to the data we find that there is a good level of fitting with a very small difference of ~ 0.5% between training and testing sets, with an accuracy of about 90% for the test set, so the model fits the data at a suitable level.

The model predicts the probability of signing a long-term deposit to be about 11%. For this model we fit cubic splines to the numerical data which helped to smooth the fit between data points and produce an accurate model.

```
traindat_bank$y <- as.numeric(traindat_bank$y, levels = c(0, 1)) - 1
testdat_bank$y <- as.numeric(testdat_bank$y, levels = c(0, 1)) - 1

gam_bank <- gam(y ~ s(age, bs = "cr", k = 5) + s(duration, bs = "cr", k = 5) + s(campaign, bs = "cr", k = 5) + s(previous, bs = "cr", k = 5) + s(pdays, bs = "cr", k = 5) + job + marital + education + default + housing + loan  + contact + month + day_of_week + contract_completed + poutcome, data = traindat_bank, family = binomial)

pred_gam_bank <- predict.gam(gam_bank, newdata=testdat_bank, type="response")
pred_gam_bank_train <- predict.gam(gam_bank, newdata=traindat_bank, type="response")

mean_prob_gam_bank <- mean(pred_gam_bank)
print(paste("The probability of signing a long term deposit using a generalised additive model is", mean_prob_gam_bank))

pred_gam_bank_train[which(pred_gam_bank_train<=0.5)]=0
pred_gam_bank_train[which(pred_gam_bank_train>0.5)]=1
pred_gam_bank_train <- as.factor(pred_gam_bank_train)
accuracy_gam_bank_train <- sum(pred_gam_bank_train==traindat_bank$y) / length(pred_gam_bank_train)
print(paste("The probability of signing a long term deposit using training
```

```
data and a generalised additive model has an accuracy of",
accuracy_gam_bank))

pred_gam_bank[which(pred_gam_bank<=0.5)]=0
pred_gam_bank[which(pred_gam_bank>0.5)]=1
pred_gam_bank<-as.factor(pred_gam_bank)
accuracy_gam_bank<-sum(pred_gam_bank==testdat_bank$y) / length(pred_gam_bank)
print(paste("The probability of signing a long term deposit using testing
data and a generalised additive model has an accuracy of",
accuracy_gam_bank))
```

The probability of signing a long-term deposit using a generalised additive model is **0.112278587574577.**

The probability of signing a long term deposit using training data and a generalised additive model has an accuracy of **0.909769643995265.**

The probability of signing a long-term deposit using testing data and a generalised additive model has an accuracy of **0.904133284777859.**

f).

The generalized additive model and the logistic regression both had very similar predictions when it came to finding the probability of signing a long-term deposit both giving predictions of ~ 11%. Both models had test accuracies of ~ 90% as well. The generalized additive model however had a smaller difference between accuracy of training and testing sets implying a better overall fit.

The random forest didn't perform as well in this question, it gave a noticeable difference in its predictions giving a probability to sign a long-term contract of ~ 8%. Furthermore the difference in accuracy of training and testing sets was ~ 5% implying that the fit was also worse compared to the generalized additive model and the logistic regression.

g).

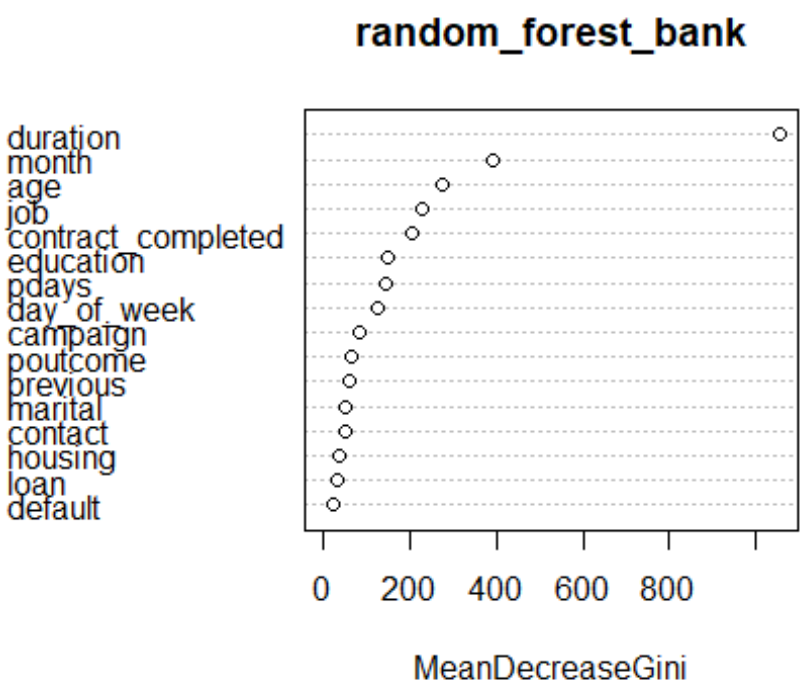
The three most important variables indicated by the random forest model was duration, month and age. We obtain this information by using the varImpPlot() graph which shows the mean decrease in gini impurity for all of the variables.

The predictions made by the logistic model that used 3 variables was almost exactly the same to the model which used all of the variables, both predicting the probability of signing a long term deposit to be ~ 11%. The fit was very slightly worse using only 3 variables giving a difference of accuracy between training and testing sets of ~ 0.9% and a testing accuracy of again ~ 90%. Overall the logistic regression with 3 of the most important variables captures exactly what is generated by the logistic regression that uses all of the variables.

This approach is useful when we are given a data set with a lot more different variables, computational time may be much longer, furthermore it may not be useful to analyse the fit using all the variables. If we are given a task by a company to analyse trends in a data set and make predictions so the company can make improvements to its work, the company would benefit

from learning about the influence of the most important factors rather than trying to improve its work by improving all factors.

```
varImpPlot(random_forest_bank)
```




```
traindat_bank$y <- factor(traindat_bank$y, levels = c(0, 1))
testdat_bank$y <- factor(testdat_bank$y, levels = c(0, 1))

logistic_bank = glm(y ~ month + duration + age, family = binomial(logit),
data=traindat_bank)

predicted_probabilities_train <- predict(logistic_bank,
newdata=traindat_bank, type = "response")
predicted_probabilities_test <- predict(logistic_bank, newdata=testdat_bank,
type = "response")
mean_prob_logistic_bank <- mean(predicted_probabilities_test)
print(paste("The probability that a customer will sign up to a long term
deposit using a logistic regression model that only uses the three most
important variables is", mean_prob_logistic_bank))

predicted_class_train <- ifelse(predicted_probabilities_train > 0.5, 1, 0)
predicted_class_test <- ifelse(predicted_probabilities_test > 0.5, 1, 0)

predicted_class_train <- factor(predicted_class_train, levels = c(0, 1))
predicted_class_test <- factor(predicted_class_test, levels = c(0, 1))

cm_train <- confusionMatrix(predicted_class_train, traindat_bank$y)
```

```
cm_test <- confusionMatrix(predicted_class_test, testdat_bank$y)

accuracy_log_train_bank <- cm_train$overall['Accuracy']
accuracy_log_test_bank <- cm_test$overall['Accuracy']

print(paste("From the train data the probability that a customer will sign up
to a long term deposit using a logistic regression that only uses the three
most important variables has an accuracy of", accuracy_log_train_bank))

print(paste("From the test data the probability that a customer will sign up
to a long term deposit using a logistic regression that only uses the three
most important variables has an accuracy of", accuracy_log_test_bank))
```

The probability that a customer will sign up to a long-term deposit using a logistic regression model that only uses the three most important variables is **0.112680946441499.**

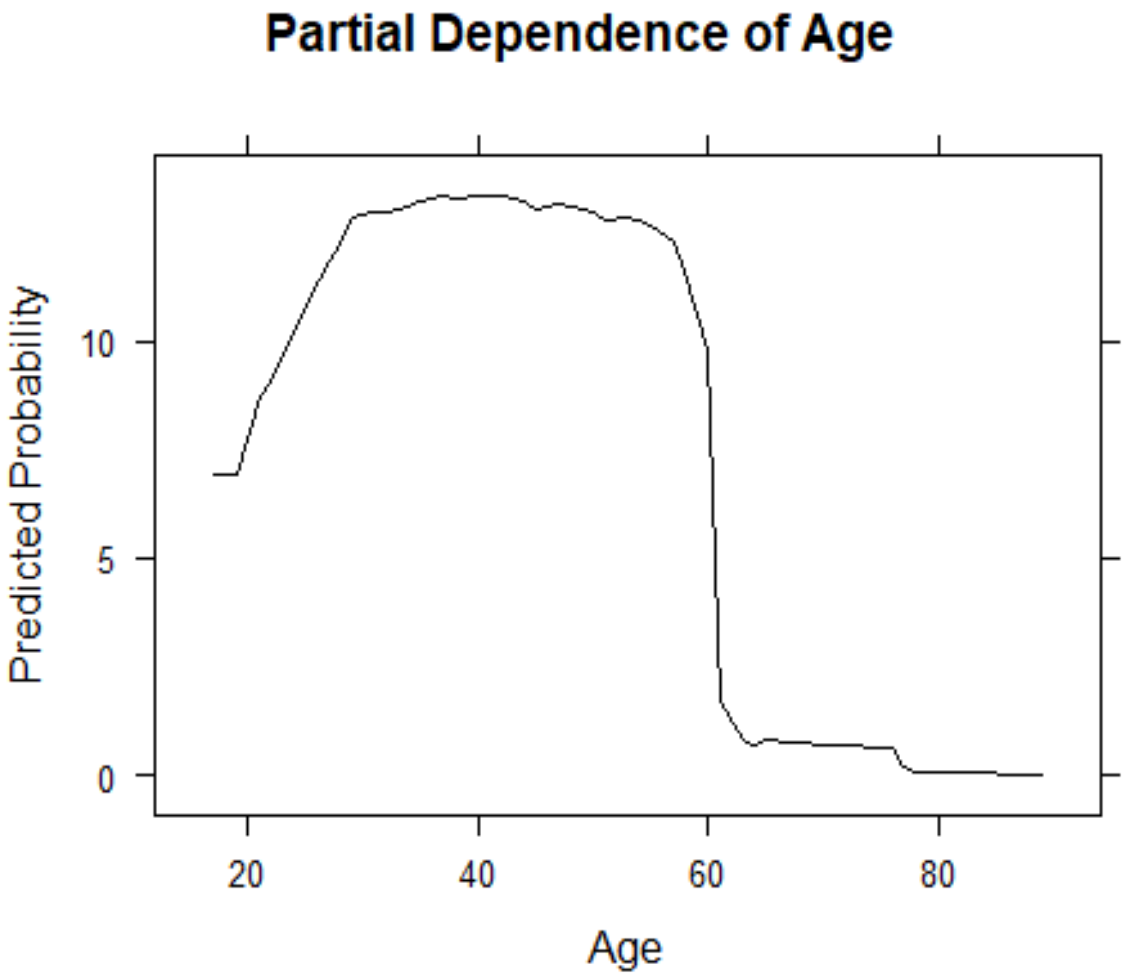
From the train data the probability that a customer will sign up to a long-term deposit using a logistic regression that only uses the three most important variables has an accuracy of **0.900482563962487.**

From the test data the probability that a customer will sign up to a long-term deposit using a logistic regression that only uses the three most important variables has an accuracy of **0.891478514202476.**

h).

```
pdp_rfst_age <- partial(random_forest_bank, pred.var = "age", train =
traindat_bank)
plotPartial(pdp_rfst_age, main = "Partial Dependence of Age", xlab = "Age",
ylab = "Predicted Probability")
```

[5]


Partial Dependence of Age
Predicted Probability
10
5
0
20
40
60
80
Age

References:

Note: Much of the work was referenced from G. Aivaliotis: Practical 1 (2024), Practical 2 (2024), Practical 3 (2022), all available on Minerva -> Learning Resources -> Practicals.

[1] B. Radečić, 'Complete guide to Gradient Boosting and XGBoost in R', *Appsilon.com*. [Online]. Available: https://www.appsilon.com/post/r-xgboost. [Accessed: 04-Dec-2024].

[2] 'Function reference', *Tidyverse.org*. [Online]. Available: https://ggplot2.tidyverse.org/reference/. [Accessed: 05-Dec-2024].

[3] Z. Bobbitt, 'How to create a confusion matrix in R (step-by-step)', *Statology*, 01-Apr-2021. [Online]. Available: https://www.statology.org/confusion-matrix-in-r/. [Accessed: 05-Dec-2024].

[4] M. Siavoshi, 'Step-by-step guide to logistic regression in R', *Statology*, 28-Oct-2024. [Online]. Available: https://www.statology.org/step-by-step-guide-to-logistic-regression-in-r/. [Accessed: 05-Dec-2024].

[5] *R-project.org*. [Online]. Available: https://journal.r-project.org/articles/RJ-2017-016/. [Accessed: 05-Dec-2024].